

SISTEMAS OPERACIONAIS

```
mkdir "Universidade Federal de Alfenas"  
touch "Universidade Federal de Alfenas/Turma SO 2025-02.txt"  
echo "Professor Romário Borges" > "Universidade Federal de Alfenas/Turma SO  
2025-02.txt"
```

Impasse (Deadlock)

```
cat aula06.txt
```

Na aula de Hoje, teremos:

- O conceito de impasse (deadlock);
- Condições para que haja um impasse;
- Como lidar com deadlocks;

Impasse

“Em um **impasse**, duas ou mais tarefas se encontram bloqueadas, aguardando eventos que dependem somente delas” (Maziero, 2015).

“Em um **deadlock**, os processos nunca terminam sua execução, e os recursos do sistema ficam ocupados, impedindo que outros jobs comecem a ser executados.” (Silberschatz, 2015)

Impasse



Condições para que ocorra impasse

1. **Exclusão mútua:** o acesso aos recursos deve ser feito de forma mutuamente exclusiva. Apenas uma tarefa por vez pode acessar cada região crítica por vez.
2. **Condição de Posse e Espera:** uma tarefa pode solicitar o acesso a outros recursos sem ter de liberar os recursos que já detém.
3. **Não-preempção:** um processo só libera os recursos que detém quando assim o decidir (ou seja, o sistema operacional não retira à força os recursos alocados às tarefas).
4. **Espera Circular:** Deve haver uma lista circular de dois ou mais processos, cada um deles esperando por um processo de posse do membro seguinte da cadeia.

Condições para que ocorra impasse

Pensando no problemas do Jantar dos filósofos:

Condição	Significado	Exemplo no jantar	
1 Exclusão mútua	o recurso só pode ser usado por um processo	o garfo só pode ser segurado por um filósofo	
2 Posse e espera	um processo segura um recurso e espera outro	o filósofo segura o garfo da esquerda e espera o da direita	
3 Não preempção	o recurso não pode ser retirado à força	o filósofo não solta o garfo	
4 Espera circular	há um ciclo de processos esperando entre si	cada filósofo espera o garfo do vizinho	

Qual código
apresenta
uma falha?

```
typedef int semaphore;  
semaphore resource_1;  
semaphore resource_2;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}  
  
void process_B(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

(a)

```
semaphore resource_1;  
semaphore resource_2;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}  
  
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources( );  
    up(&resource_1);  
    up(&resource_2);  
}
```

(b)

DEADLOCK!!!

```
typedef int semaphore;  
semaphore resource_1;  
semaphore resource_2;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}  
  
void process_B(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

(a)

```
semaphore resource_1;  
semaphore resource_2;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}  
  
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources( );  
    up(&resource_1);  
    up(&resource_2);  
}
```

(b)

Processo

Recurso

FIGURA 6.3

Grafos de alocação de recursos. (a) Processo de posse de um recurso. (b) Solicitação de um recurso. (c) Impasse.

A

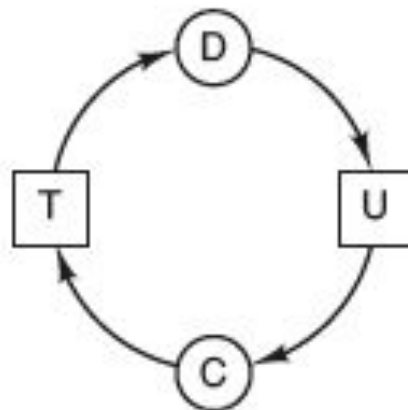
R

(a)

S

B

(b)



(c)

Como lidar com
Deadlocks?

Como lidar com Impasse?

1. **Ignorar**: Se você o ignorar, quem sabe ele ignore você.
2. **Detecção e Recuperação**: Deixe-os ocorrer, detecte-os e tome as medidas cabíveis.
3. **Evitar**: Evitar dinamicamente pela alocação cuidadosa de recursos.
4. **Prevenção**: ao negar estruturalmente uma das quatro condições.

Ignorar

Quando a **probabilidade** de ocorrer, a **frequência** e o **impacto** são baixos.



A abordagem mais simples é o **algoritmo do avestruz**: enfie a cabeça na areia e finja que não há um problema.

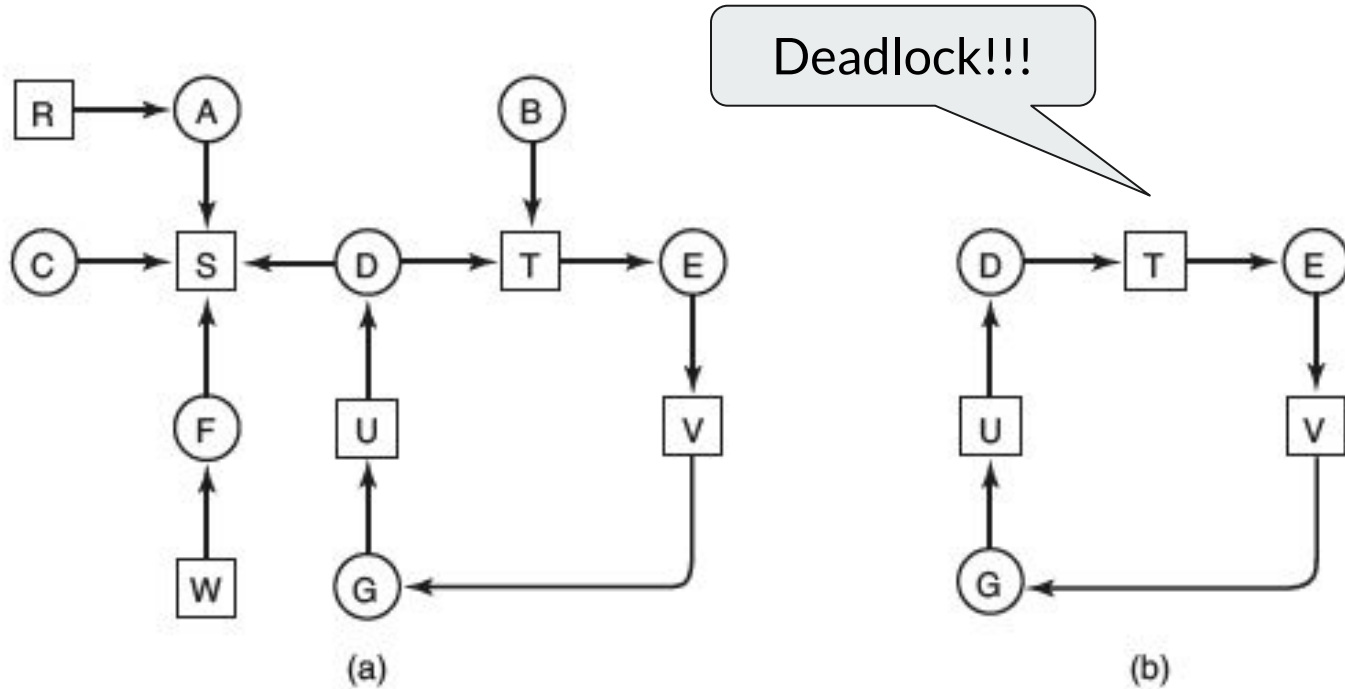
A maioria dos SOs o implementam. Mas imagina quais seriam os problemas caso um sistema hospitalar ou militar resolvesse resolver os problemas dessa maneira?

Detecção e recuperação

Detecta que um deadlock ocorreu, **detecta** os processos envolvidos no deadlock e **corrige** o problema.

O **algoritmo de busca em profundidade** pode ser utilizado para detecção de ciclos em grafos (se existe um deadlock na interação entre processos/recursos)

Detecção e recuperação



Detecção e recuperação

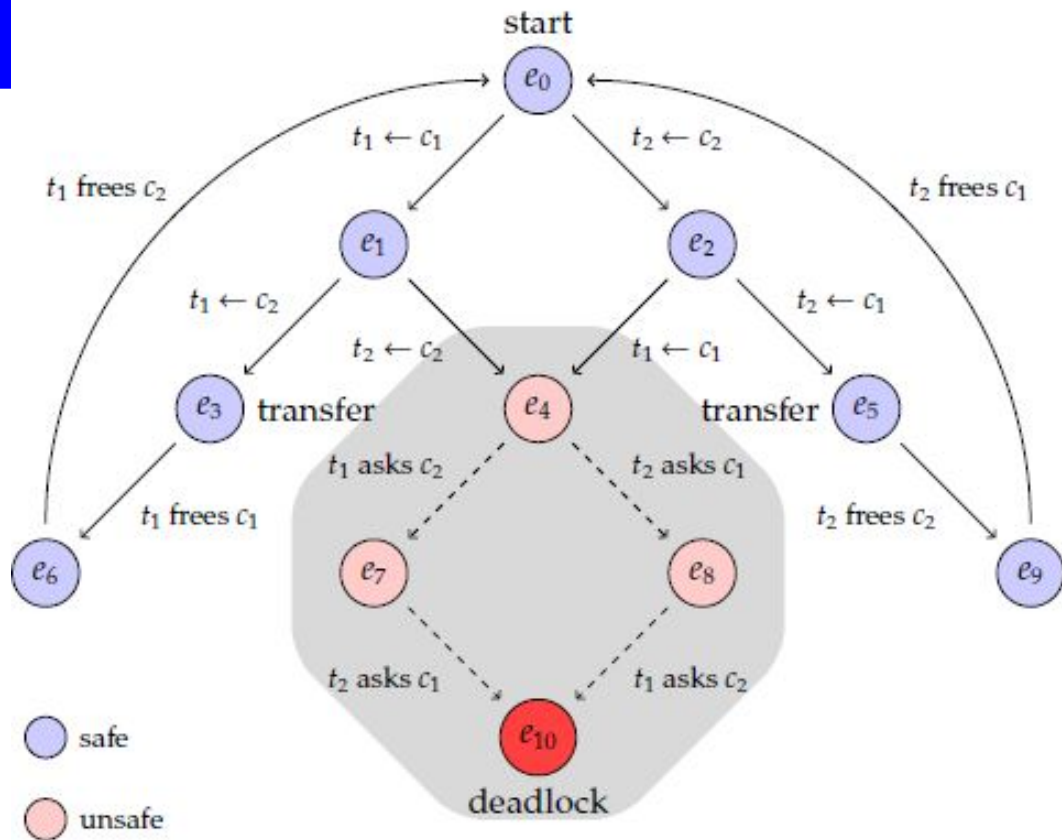
Como a recuperação pode ser feita?

- **Recuperação por meio de Preempção:** Retira um processo e fornece outro (de modo imperceptível);
- **Recuperação por Retrocesso:** **Salva os estados periodicamente.** Um processo é bloqueado, e após ser liberado o recurso é utilizado.
- **Recuperação por Eliminação:** **Mata** os processos até acabar com o deadlock;

Evitar

Acompanhar a alocação dos recursos e verificar quais tem **estado seguro** (evitam deadlock) ou não tem **estado inseguro** (podem causar deadlock).

É necessário saber com antecedência o comportamento dos processos. Isso é de fácil realização?



Evitar

Criado por Dijkstra em 1965, o **algoritmo do banqueiro** faz uma analogia entre as tarefas de um sistema e os clientes de um banco, tratando os recursos como créditos emprestados às tarefas para a realização de suas atividades.



Prevenir

As técnicas de **prevenção de impasses** buscam garantir que **impasses nunca possam ocorrer** no sistema. Se ao menos uma das quatro condições de ocorrência de impasses for quebrada, os impasses não poderão ocorrer.

Prevenir

Exclusão mútua: se não houver exclusão mútua no acesso a recursos, não poderão ocorrer impasses. Ex: um servidor de impressão (*spooling*). A técnica de *spooling* previne impasses envolvendo as impressoras, mas não é facilmente aplicável a certos tipos de recurso, como arquivos em disco e áreas de memória compartilhada.

Posse e Espera: caso as tarefas usem apenas um recurso de cada vez, solicitando-o e liberando-o logo após o uso, impasses não poderão ocorrer. Ou requerer todos os processos antes de executar; Essa abordagem poderia levar as tarefas a reter os recursos por muito mais tempo que o necessário para suas operações, degradando o desempenho do sistema.

Prevenir

Não-preempção: Se um processo tomar a força um recurso de outro processo que tinha posse do mesmo, evitamos os deadlocks. No entanto, é de difícil aplicação sobre recursos como arquivos ou áreas de memória compartilhada, porque a preempção **viola a exclusão mútua** e pode provocar inconsistências no estado interno do recurso.

Espera Circular: Ao prevenir a formação de ciclos (de uma cadeia de dependência), impasses não poderão ocorrer. A estratégia mais simples para prevenir a formação de ciclos é **ordenar** todos os recursos do sistema de acordo com uma ordem global única, e forçar as tarefas a solicitar os recursos obedecendo a essa ordem.

Na aula de Hoje, aprendemos:

- O conceito de impasse (deadlock);
- Condições para que haja um impasse;
- Como lidar com deadlocks;

Referencial Bibliográfico

- Maziero, Carlos Alberto – ***Sistemas Operacionais: Conceitos e Mecanismos***
- Tanenbaum, Andrew S.; Bos, Herbert – **Sistemas Operacionais Modernos** (tradução de Modern Operating Systems, 4ª edição);
- Silberschatz, Abraham; Galvin, Peter Baer; Gagne, Greg – ***Fundamentos de Sistemas Operacionais*** (tradução de *Operating System Concepts*, 9ª edição)

SISTEMAS OPERACIONAIS

Exit